BME INSTITUTE - TALLER B1-T1

Multi-Source Portfolio Analytics and Monte Carlo Toolkit

Proposed GitHub implementation plan

Proposal intent

I propose a modular Python toolbox that fully covers the mandatory workshop requirements and adds a limited number of quantitative extensions with clear financial value. The core repository should remain simple to run, independent of paid market-data entitlements, and extensible through provider and model interfaces.

GitHub delivery deadline: 12 November 2026, 18:00

Working proposal - 6 October 2026

1. Proposed scope and design principle

The workshop is primarily a software-engineering exercise applied to market data and portfolio analytics. My proposal is therefore to make the architecture and standardisation layer the centre of the repository, and to add quantitative sophistication only where it remains modular and does not become a dependency for the basic workflow.

Core rule

The repository should work end-to-end with historical prices and historical volatility alone. Option-implied volatility, rates-option surfaces and local-volatility modelling are optional calibration layers, never prerequisites for running the portfolio simulation.

Mandatory scope I would cover

- A clean /src package, README, dependency file, .gitignore and reproducible execution path.
- A multi-provider market-data layer with identical internal outputs regardless of source.
- Batch extraction of N time series from a list of symbols or instrument specifications.
- DataClasses for price series and portfolio objects, including automatic basic statistics.
- Cleaning, NaN handling and calendar alignment across markets and asset classes.
- A parameterised Monte Carlo method available for an individual asset and for the portfolio.
- A Markdown .report() method and a .plots_report() method with useful diagnostics.
- A formal architecture/UML diagram and a repository structure that makes extensions easy.

Quantitative extensions I would add

- A realistic multi-asset demonstration portfolio rather than an equity-only toy example.
- Correlated Gaussian Monte Carlo as the baseline model and Student-t Monte Carlo as a fat-tail alternative.
- Historical volatility as the universal fallback calibration, plus optional option-implied volatility where suitable chains are available.
- Additional risk measures such as drawdown, skewness, kurtosis, VaR and Expected Shortfall.
- An optional volatility-surface module that can be developed independently from the core repository.

2. Core architecture

I would separate data acquisition, normalisation, financial objects, analytics and simulation. The Monte Carlo engine should never need to know whether a price came from Yahoo, Alpha Vantage or another source; it should consume standardised internal objects.

1	**Provider adapters**	Yahoo / Alpha Vantage / FRED / ECB / MarketData.app
2	**Standardisation**	Common internal schema, metadata and calendar alignment
3	**DataClasses**	PriceSeries, RateSeries, Portfolio; optional OptionSurface
4	**Analytics & simulation**	Statistics, correlation, Gaussian / Student-t Monte Carlo
5	**Outputs**	Markdown report, plots, diagnostics and risk metrics

Suggested repository shape

```
src/
  data/
    base.py
    yahoo.py
    alpha_vantage.py
    fred.py
    ecb.py
    marketdata_options.py
  models/
    price_series.py
    rate_series.py
    portfolio.py
  analytics/
    statistics.py
    risk.py
    correlation.py
  simulation/
    base.py
    gaussian.py
    student_t.py
    local_vol.py        # optional extension
  volatility/
    historical.py
    implied.py          # optional extension
    surface.py          # optional extension
  reporting/
    reports.py
    plots.py
tests/
```

```
README.md
pyproject.toml or requirements.txt
```

Internal objects

Object	Purpose
PriceSeries	Historical market prices plus metadata, returns, mean and volatility.
RateSeries	Sovereign yields / policy rates / macro rates retrieved from FRED or ECB.
Portfolio	Assets, weights, aligned returns, aggregate analytics and Monte Carlo method.
OptionSurface	Optional object for strike/maturity option data, implied volatilities and Greeks.

Design objective

New providers and new simulation models should be added by implementing a common interface, without changing the Portfolio class or the existing analytics logic.

3. Data-provider strategy

I would use several providers, but I would not optimise for the largest possible number of APIs. The important point is to demonstrate that heterogeneous sources can be normalised behind the same interface and that the core repository continues to work if one provider is replaced.

Provider	Main role	Proposed use
Yahoo / yfinance	Equities, indices, ETFs	Primary historical-price source; simple and convenient for the core workflow.
Alpha Vantage	Equities / alternative data	Second independent provider and useful demonstration of provider interchangeability.
FRED	US rates and macro	US Treasury yields and additional explanatory factors.
ECB Data API	EUR rates and macro	Euro-area / sovereign yield data and EUR-related macro series.
MarketData.app	US listed options	Primary optional source for option chains, implied volatility and Greeks.
Massive / Polygon	Options / market data	Optional secondary option provider or source for option-price cross-checks.
Interactive Brokers	Broad market data	Optional adapter only; the repository should not depend on IBKR subscriptions or entitlements.

Common provider interface

```
get_price_series(instrument, start, end)
get_rate_series(series_id, start, end)
get_option_chain(underlying, expiry=None)      # optional capability
get_metadata(instrument)

All provider-specific responses -> normalised internal DataClasses
```

Important boundary

FRED and ECB are ideal for sovereign yields and macro rates, but a continuous "10-year bond price" is not the natural object provided by those APIs. For portfolio valuation I therefore favour tradable fixed-income price series; yields remain a complementary risk-factor layer.

4. Demonstration portfolio and fixed-income treatment

For the main example I propose a 70% equity / 30% sovereign fixed-income portfolio. The exact tickers can be finalised later; the purpose is to demonstrate multiple asset classes, different market calendars, cross-asset correlation and a financially meaningful Monte Carlo exercise.

Illustrative sleeve	Weight	Reason
S&P 500 proxy	30%	Broad US equity beta
Nasdaq-100 proxy	15%	Growth / technology exposure
Semiconductor or AI ETF	10%	Sector concentration
1-2 AI / semiconductor stocks	15%	Single-name risk and stock-specific option volatility
US Treasury ETF	15%	US sovereign duration exposure using a directly observable price series
EUR sovereign ETF	15%	EUR sovereign duration exposure using a directly observable price series

Why I prefer fixed-income ETF prices for the core

- The workshop is built around price-series objects, so a Treasury ETF or sovereign-bond ETF enters the same PriceSeries pipeline as an equity or index proxy.
- Historical returns, volatility and correlation are then computed consistently across the entire portfolio.
- This avoids turning the core assignment into a cash-bond pricing library with coupon schedules, accrued interest, clean/dirty pricing, day-count conventions and yield-curve bootstrapping.
- If a robust market provider exposes an individual sovereign-bond price series, the architecture can also accept it; the ETF route is simply the most reliable default.

Role of sovereign yields

I would still retrieve US and EUR sovereign yields from FRED and ECB. They are useful explanatory risk factors, can be shown in the report alongside bond-ETF performance, and leave the door open to a later duration/convexity or yield-curve extension.

Core fixed-income implementation

Portfolio value is driven by market prices of Treasury / sovereign-bond ETFs. FRED and ECB yields are additional analytical series, not the source of the base portfolio price.

5. Volatility calibration: historical first, implied as an extension

Every asset should have a volatility input even if no option market is available. For that reason, historical volatility is the default calibration for the Monte Carlo engine. Option-implied volatility is an optional, forward-looking alternative wherever liquid option data can be retrieved.

Exposure	Default volatility	Optional market-implied calibration
S&P 500	Historical SPY / index-proxy returns	SPX or SPY option surface
Nasdaq-100	Historical QQQ / index-proxy returns	NDX or QQQ option surface
AI / semiconductor ETF	Historical ETF returns	ETF options if the chain is sufficiently liquid
AI single stock	Historical stock returns	Its own listed option surface
US Treasury ETF	Historical ETF returns	ETF options if useful; advanced proxy from options on US Treasury futures
EUR sovereign ETF	Historical ETF returns	Advanced proxy from options on Euro-Bund or related sovereign futures

Rates-implied volatility as an optional plug-in

For sovereign fixed income, I would not require an option surface on the ETF itself. A more institutional extension is to retrieve implied volatility from listed options on sovereign-bond futures and use it as a proxy for the corresponding duration risk:

```
US Treasury sleeve
  Treasury ETF price -> portfolio P&L
  optional: 10Y Treasury futures options -> rates implied-vol surface

EUR sovereign sleeve
  EUR sovereign ETF price -> portfolio P&L
  optional: Euro-Bund futures options -> EUR core-rates implied-vol surface
```

The mapping is intentionally optional because futures-option market data can be more difficult or expensive to source programmatically. If the surface is unavailable, the simulation simply falls back to historical ETF volatility; the core workflow remains unchanged.

Equity option surfaces

For SPX/SPY, NDX/QQQ and selected AI single stocks, the option layer can retrieve strike, maturity, bid/ask, implied volatility and Greeks. Each underlying has its own surface. The surface is a calibration input; it is not merged across different underlyings.

Fallback rule

Historical volatility must always be available. Implied volatility enriches the simulation when data quality and access are adequate, but a missing option chain must never break the portfolio engine.

6. Monte Carlo design

I propose two core stochastic models behind the same simulation interface. Both should use the portfolio correlation structure so that assets are simulated jointly rather than as independent paths.

Model	Status	Purpose
Gaussian Monte Carlo	Baseline	Correlated normal shocks using historical means, volatilities and correlation/covariance.
Student-t Monte Carlo	Main quantitative extension	Correlated fat-tail shocks, with the scale adjusted so volatility remains comparable to the Gaussian case.
Local-vol Monte Carlo	Optional plug-in	Uses an option-implied volatility surface and, if developed, a local-volatility transformation; not required for the core submission.

Gaussian baseline

For each asset, the simulator uses an expected return, volatility and the cross-asset correlation matrix. Correlated shocks can be generated through a Cholesky decomposition. The simulation should be configurable by number of paths, horizon, time step, return assumption and volatility source.

Student-t extension

The Student-t model is particularly useful because financial returns exhibit heavier tails than a Gaussian model. I would either estimate the degrees of freedom from historical returns or expose it as a parameter. The shocks should be standardised so that a 20% input volatility means approximately the same variance under both models; this makes the tail comparison meaningful.

Portfolio aggregation

Each Monte Carlo path generates simulated values for all assets, applies the portfolio weights and produces a terminal portfolio value and P&L. The same engine can then calculate the probability of loss, confidence intervals, VaR and Expected Shortfall.

Model-measure distinction

Historical / Student-t simulations are intended for real-world portfolio-risk scenarios. If a local-vol model is added from option prices, it should be presented separately as a market-implied / risk-neutral extension rather than as a direct return forecast.

7. Analytics, reporting and GitHub quality

The final repository should demonstrate value beyond data download. I propose a compact but useful analytics layer that automatically turns the standardised time series into portfolio diagnostics and reproducible outputs.

Automatic analytics

- Daily and annualised return statistics and volatility.
- Correlation and covariance matrices after calendar alignment.
- Skewness, kurtosis and maximum drawdown.
- Portfolio return / volatility and, where useful, risk contribution by sleeve.
- VaR and Expected Shortfall for the simulated portfolio distribution.
- Comparison of Gaussian and Student-t tail behaviour.

report() and plots_report()

Method	Proposed output
report()	Markdown summary of portfolio composition, data quality, statistics, assumptions, simulation parameters, risk metrics and warnings.
plots_report()	Cumulative performance, correlation heatmap, return histogram, Gaussian vs Student-t comparison, Monte Carlo fan chart, terminal-value distribution and drawdown.

Repository quality requirements

- No API keys committed to GitHub; use environment variables and provide an .env.example where relevant.
- Tests for provider normalisation, return calculations, portfolio weights and simulation reproducibility.
- Clear error handling for missing tickers, unavailable endpoints, empty option chains and rate limits.
- Deterministic README examples and documented provider/model interfaces so optional modules can be added without changing the core.

Implementation priority

Layer	Scope
Priority 1 - Core	End-to-end multi-provider prices -> normalisation -> Portfolio -> Gaussian MC -> report/plots. This must be robust first.
Priority 2 - Quant enhancement	Student-t Monte Carlo, richer risk metrics and the 70/30 multi-asset demonstration.
Priority 3 - Optional plug-ins	Equity option IV, sovereign-futures IV proxies, own IV solver / surface, local vol and optional IBKR or rates-pricing extensions.